40_CI_CD.md

UNPLND CI/CD & Release Pipeline

Version 1.0

Priority: HIGH

Status: Foundation

Purpose

Automate everything that should never depend on memory.

Every commit.

Every build.

Every release.

Should follow the same predictable process.

The goal is confidence.

Not complexity.

Philosophy

Small team.

Professional workflow.

Simple automation.

Reliable releases.

Tech Stack

Source Control

GitHub

Branch Protection

Enabled

CI

GitHub Actions

Deployment

Firebase

TestFlight

Google Play Internal Testing

Branch Strategy

main

Production-ready only.

develop

Active development.

feature/*

One feature per branch.

hotfix/*

Critical production fixes.

release/*

Optional for major releases.

Never commit directly to main.

Pull Request Rules

Every PR requires:

Passing CI

No merge conflicts

Successful build

At least one review

Meaningful description

Commit Convention

Examples

feat: add planner onboarding

fix: resolve route regeneration bug

refactor: simplify album model

docs: update API specification

style: improve spacing

test: add planner widget tests

Consistent history matters.

CI Pipeline

On every Pull Request

↓

Install dependencies

↓

Analyze code

↓

Format verification

↓

Unit tests

↓

Widget tests

↓

Build app

↓

Report results

Merge only if everything passes.

Quality Gates

Required

Flutter analyze

No formatting issues

All tests passing

No critical warnings

Successful build

Optional

Coverage threshold

Performance benchmarks

Release Pipeline

Tag created

↓

Build iOS

↓

Build Android

↓

Upload artifacts

↓

Create release notes

↓

Deploy to TestFlight

↓

Deploy to Internal Testing

↓

Manual approval

↓

Production

Secrets

Store only in GitHub Secrets.

Never commit

API keys

Firebase keys

Signing keys

Tokens

Passwords

Rotate secrets regularly.

Code Quality

Use

flutter analyze

dart format

flutter test

before every merge.

Automation should enforce standards.

Dependency Management

Review dependencies monthly.

Remove unused packages.

Pin critical versions.

Avoid unnecessary libraries.

Versioning

Semantic Versioning

Major.Minor.Patch

Examples

1.0.0

1.1.0

1.1.1

Tag every production release.

Build Types

Development

Staging

Production

Separate configuration for each.

Never mix environments.

Testing

Required before release

Unit Tests

Widget Tests

Integration Tests

Smoke Test

Manual QA

Rollback

Every production release must support rollback.

Keep previous release artifacts.

Document rollback steps.

Monitoring

Automatically upload symbols.

Enable Crashlytics.

Enable Performance Monitoring.

Monitor release health.

Notifications

CI failures

↓

GitHub

Critical release failures

↓

Email

Optional

Slack/Discord integration

Release Notes

Automatically generated.

Edited manually before production.

Always include

New features

Fixes

Known issues

Documentation

Every major feature update

↓

Update documentation

↓

Update PRD if needed

↓

Update roadmap if required

Documentation evolves with the product.

Security Checks

Dependency scanning

Secret scanning

Basic vulnerability review

Automated where possible.

Performance Checks

Future

Startup time

Build size

Frame rendering

Regression tracking

Backup

Daily database backup.

Storage backup.

Configuration backup.

Recovery plan documented.

Delight Checklist

✓ Reliable.

✓ Automated.

✓ Predictable.

✓ Easy to understand.

✓ Easy to maintain.

DO NOT

Do NOT deploy directly from feature branches.

Do NOT skip tests.

Do NOT ignore failing CI.

Do NOT manually edit production data.

Do NOT release without changelog.

Golden Rule

Automation should remove repetitive work,

not remove human judgment.

Production releases always require intentional approval.

Claude Instructions

Never merge code that fails the pipeline.

Never bypass CI.

Whenever introducing a new tool,

keep the workflow simple enough that two founders can manage it comfortably.

Optimize for long-term maintainability.

END OF 40_CI_CD.md